Lambda Functions

Anonymous Power in C++

The Modern Way to Write Inline Logic

[CAPTURE] (PARAMS) → RET { BODY }

1. Introduction to Anonymous Functions

A **Lambda Function** (introduced in C++11) is a small, unnamed function that you can define directly inside your code, usually as an argument to another function or for a quick, one-time task.

Why use them?

- **Conciseness:** Avoid the "boilerplate" of creating a full, named function in a separate space.
- **Locality:** Keep the logic close to where it is actually used.
- **STL Integration:** They are the primary tool for algorithms like `std::sort` or `std::for_each`.

> **Think of it like:** A "Sticky Note" function. You don't need to write a whole manual (Full Function) just to remember a tiny task.

2. The Syntax Map

While normal functions have a name and return type first, Lambdas start with a "Capture Clause."

[Capture] (Parameters) → Return { Body };

```
auto myLambda = []() {
    cout << "I have no name!";
};

myLambda(); // Executes the lambda
```

The Anatomy:

- **[ ]** : Capture Clause (Accessing outside variables).
- **( )** : Parameter List (Input values).
- **→** : Trailing Return Type (Optional).
- **{ }** : Function Body (The Logic).

3. Parameters & Return Types

Lambdas behave exactly like functions when it comes to inputs and outputs. You can pass values just as you would with `int add(int a, int b)`.

Example with Parameters:

```
auto sum = [](int a, int b) {
    return a + b;
};

cout << sum(10, 20); // Result: 30
```

Manual Return Types:

Usually, the compiler "deduces" the return type. If your logic is complex, you can specify it manually using `-> type`.

```
auto calculate = [](int x) -> double {
    if (x > 0) return x * 1.5;
    return 0.0;
};
```

4. Data Access: Capture by Value

One of the most powerful features of Lambdas is their ability to "capture" variables that exist in the surrounding code.

The [var] syntax:

When you put a variable name inside the brackets, the lambda makes a **copy** (a clone) of that variable at that specific moment.

```
int multiplier = 2;

auto timesTwo = [multiplier](int n) {
    return n * multiplier; // Uses the copy
};

multiplier = 10; // This does NOT affect the lambda!
cout << timesTwo(5); // Still Outputs: 10
```

5. Data Access: Capture by Reference

If you want the lambda to work with the **live original variable**, you use the & symbol inside the capture clause.

```
int counter = 0;

auto increment = [&counter]() {
    counter++; // Directly modifies the original variable
};

increment();
increment();

cout << counter; // Outputs: 2
```

Mass Captures:

- `[=]` : Capture everything in the area **by value**.
- `[&]` : Capture everything in the area **by reference**.

6. Lambdas and the STL

The Standard Template Library (STL) algorithms were practically made for Lambdas. Instead of writing a helper function to sort numbers, you do it on one line.

Example: std::sort

```
vector v = {4, 1, 3, 2};

sort(v.begin(), v.end(), [](int a, int b) {
    return a > b; // Sort in descending order
});
```

This "Inline Comparator" makes the code much more readable because the sorting logic is right next to the sorting command.

7. Passing Lambdas as Arguments

You can pass a lambda function as a parameter to another normal function. In modern C++, we use `std::function` (from the headers) to handle this.

```
#include

void executor(std::function func) {
    func(100);
}

int main() {
    executor([](int val) {
        cout << "Executing with: " << val;
    });
}
```

8. Limitations & Warnings

1. Complexity Limit

If a lambda exceeds 10 lines of code, it should probably be a named function. Lambdas are meant for "glance-able" logic.

2. Debugging Difficulty

Anonymous functions don't have names in stack traces. If a lambda crashes, identifying it in a large project can be difficult.

3. Memory Risk

Capturing by reference (`[&]`) is dangerous if the lambda lives longer than the original variable. This leads to "Dangling References."

9. Best Practice Guidelines

Rule 1: Keep it Small. Use lambdas for simple data transformations and predicates.

Rule 2: Capture Explicitly. Instead of using `[=]`, specify exactly what you need (e.g., `[x, y]`). This makes the code clearer and faster.

Rule 3: Use 'auto'. Always use `auto` to store a lambda because its exact type is complicated and generated by the compiler.

10. Summary & Skill Lab

Lambdas shifted C++ from a strictly object-oriented language to one that embraces functional programming patterns.

The 30-Second Recap:

- **Anonymous:** No name required.
- **Capture:** Can "eat" surrounding variables.
- **Inline:** Defined where they are used.

Practical Challenge:

The Filter: Given a `vector` of scores, use `std::remove_if` with a lambda for a capture variable `threshold = 50` to remove all scores below 50.

MODULE COMPLETE: C++ LAMBDA ARCHITECTURE